HAZMAT SAFETY DATA SHEET

0xc2292af8b51109a27a4b4e28e9cf9f3aede4190c (verified as ClausTokenDealEscrow)

Read 11,566 bytes of deployed bytecode and checked it against 16 registered hazards. 3 hazards are present, reproduced against this very contract.

What this is: It is a plain contract on mainnet, read from its bytecode.
What matched: 3 of the 16 registered hazards are present: HZ-001 (ordering), HZ-009 (ordering), HZ-005 (stranded-value).
How sure: HZ-001, HZ-009, HZ-005 were reproduced against this exact contract, not a lookalike.
The sharpest one: HZ-001: A still-valid action is destroyed by a permissionless trigger in the same block
What is at stake: one deal amount, which is the worker's entire payment on the affected deal.
What to do about it: Bound the still-valid exit by the same clock as the permissionless trigger, or document which exit is authoritative after the window.
Also: Define the authoritative exit for the overlap, or bound both paths by the same clock.
Also: Add a sweep for stray value, or document that it is donated.

ASSERTION LEVEL: PATTERN MATCH ONLY; CONFIRMATION RUNS ARE SEPARATE
A match is a candidate, not a clearance: this sheet fingerprints a contract against registered hazard shapes, it does not audit it. A clean sheet is not the same as safe.

ANSWERS TO THE USUAL QUESTIONS
read from the selectors this contract exposes, not from its docs. Every answer says what it cannot tell you.

What is this?
It is a contract verified as ClausTokenDealEscrow on mainnet. Everything below is read from the bytecode it exposes, not from its docs.
seen: no matching selector, so this is its shape
not answered here: Whether it is any good, and who wrote it.

Who is in charge?
No owner or admin function is exposed here. Either nothing is configurable, or the power sits behind a proxy or another contract this scan could not follow.
seen: no matching selector, so this is its shape
not answered here: Which of those two it is.

Can anyone else move my balance?
Only with your approval: this is an allowance-based token, so the standing risk is what you have approved, not what the contract can do on its own.
seen: 0x23b872dd transferFrom(address,address,uint256)
not answered here: What you have approved right now - that is state, not bytecode.

BOTTOM LINE
3 of 16 registered hazards are present (HZ-001, HZ-005, HZ-009). That is the fingerprint, not a verdict: somebody still has to read the code, and the sharpest one has already been reproduced against this very contract, which raises it from a shape to a demonstrated behaviour.

BEFORE YOU TRUST IT, CHECK
- Bound the still-valid exit by the same clock as the permissionless trigger, or document which exit
- is authoritative after the window.
- Add a sweep for stray value, or document that it is donated.
- Define the authoritative exit for the overlap, or bound both paths by the same clock.

HAZARDS FOUND

HZ-001 A still-valid action is destroyed by a permissionless trigger in the same block
class 2 ordering CONFIRMED ON THIS TARGET
Two exits are legal in the same block and pay opposite amounts. The transaction that lands first decides, and the losing exit cannot be repaired because the other is already dead from its own clock.
What is at stake: one deal amount, which is the worker's entire payment on the affected deal.
What to do: Bound the still-valid exit by the same clock as the permissionless trigger, or document

which exit is authoritative after the window.
Reproduced here: confirmations/HZ-001-c2292af8.json - a rebuild of the verified source and the probes in it passed. Confirming behaviour, not intent.
matched by selectors: 0xe4725ba1 0x7c7a9804 0x82cbf88b

HZ-005 Value arriving outside the accounting has no sweep
class 7 stranded-value CONFIRMED ON THIS TARGET
A contract that pays out only from internal credits accepts forced native value or direct token transfers, counts them in no total, and offers no function to recover them. Accounting stays correct, so nothing breaks and nothing notices.
What is at stake: the stranded amount itself, unbounded by any ceiling.
What to do: Add a sweep for stray value, or document that it is donated.
Reproduced here: confirmations/HZ-005-c2292af8.json - a rebuild of the verified source and the probes in it passed. Confirming behaviour, not intent.
matched by selectors: 0x70b6fbec 0xb12ae89c 0x9b40b785

HZ-009 Two permissionless exits are both legal in one block and pay opposite amounts
class 2 ordering CONFIRMED ON THIS TARGET
Inside one block a stranger trigger and the buyer exit are both valid and settle the deal differently. Block ordering alone decides who is paid, and the loser cannot repair afterwards because the other path is already closed by its own clock.
What is at stake: one deal: full price on one ordering, zero on the other.
What to do: Define the authoritative exit for the overlap, or bound both paths by the same clock.
Reproduced here: confirmations/HZ-009-c2292af8.json - a rebuild of the verified source and the probes in it passed. Confirming behaviour, not intent.
matched by selectors: 0xc9566ca9 0x7249fbb6 0xe4725ba1 0x7c7a9804

WHAT WAS READ
a match means the selectors a placard asks about are present. It does not mean reachable, and a clean sheet is not a clearance.
placards checked 16 classes 7 selectors in code 49 the register asks about 9 otherwise unaccounted 40
0x0256965f unnamed
0x06fda65d unnamed
0x0b135d3f unnamed
0x12171d83 unnamed
0x1919585b unnamed
0x23b872dd transferFrom(address,address,uint256)
0x2cd44ac3 unnamed
0x383630b7 unnamed
0x390d83ad resolve(bytes32,uint256,bytes32,uint64,bytes)
0x4584712b unnamed
0x48b44a76 runPlan(bytes32,uint32)
0x4d342432 unnamed
0x4e487b71 Panic(uint256)
0x51a6fad5 getDeal(bytes32)
0x5274afe7 unnamed
0x56891412 unnamed
0x6f20b45d unnamed
0x81cd872a unnamed
0x84b0196e eip712Domain()
0x9f040b00 unnamed
... and 4 more

NOT CHECKED AT ALL
- resolution is best effort: unknown proxy patterns, diamond facets and beacons pointing at beacons
- are labelled not followed
- reachability
- economic preconditions
- state at any particular block
- source-level reasoning
- anything not covered by a placard

THE TECHNICAL PART
target 0xc2292af8b51109a27a4b4e28e9cf9f3aede4190c

family evm
code sha256 0x879c36efa00134685fc0a1c55c7e19be0aa85602afde66b7ff93f9fb1a85e139
body bytes read 11566
metadata excluded 53 bytes (CBOR trailer, appended after compilation)
four-byte constants 49
register 16 placards v0xd7dea67cdf60b0bc
anchor epoch 3 root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c (epoch-3.json)
register digest 0xd7dea67cdf60b0bc
anchor root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c
selector index 783 names, a convenience reading and not verification

Cite the placard id, the code sha256 and the register digest. The HTML sheet is the artifact that carries its own digest; this PDF is the sam